Ficheros secuenciales y aleatorios, directorios, flujos y excepciones

1.1. Clases de gestión de ficheros y directorios

En Java, el manejo de ficheros y directorios se realiza principalmente mediante la clase **File**, que representa la ruta a un fichero o carpeta.

- Un objeto de tipo File **no es el fichero en sí**, sino una abstracción de su ruta.
- A través de esta clase podemos comprobar si existe, crearlo, borrarlo o recorrer su contenido si es un directorio.

Métodos más importantes de File:

- exists(): indica si el fichero existe.
- createNewFile(): crea un fichero vacío.
- delete(): borra el fichero o carpeta.
- mkdir(): crea un directorio.
- listFiles(): devuelve la lista de ficheros de un directorio.
- getAbsolutePath(): devuelve la ruta absoluta.

Esto permite gestionar la **estructura de directorios** desde un programa en Java.

URL: https://docs.oracle.com/javase/8/docs/api/java/io/File.html

Ejemplo:

```
File fichero = new File("datos.txt");
if (fichero.exists()) {
    System.out.println("El fichero existe.");
} else {
    System.out.println("No existe el fichero.");
}
```

1.2. Formas de acceso a un fichero. Clases más utilizadas

Un fichero puede ser accedido de dos formas principales:

Acceso secuencial:

- Los datos **se leen o escriben de manera ordenada**, de principio a fin.
- No se puede saltar directamente a una posición concreta del fichero.
- Es la forma más común y sencilla de trabajar con ficheros.
- Ejemplos:
    - leer un fichero de texto con nombres de alumnos.
    - copiar una imagen byte a byte
- En Java se implementa usando clases de **flujos secuenciales** (que veremos en detalle en el apartado 1.3):
    - **FileReader / FileWriter** (lectura/escritura de texto carácter a carácter).
    - **BufferedReader / PrintWriter** (texto, con más comodidad/eficiencia).
    - **FileInputStream / FileOutputStream** (lectura/escritura bytes binario).

Acceso aleatorio:

- Permite **saltar a una posición específica** dentro del fichero para leer o escribir desde allí.
- Es útil cuando necesitamos modificar registros concretos sin recorrer todo el fichero.
- En Java se consigue con la **clase RandomAccessFile**.
- Muy útil para ficheros binarios de gran tamaño o cuando necesitamos acceder a un registro concreto.

Clase RandomAccessFile

Constructores:

- `new RandomAccessFile(String nombreFichero, String modo)`
- El modo puede ser:
    - "r" → solo lectura.
    - "rw" → lectura y escritura.

Lectura/escritura de datos en distintos formatos:

- readInt(), readDouble(), readUTF(), ...
- writeInt(int v), writeDouble(double v), writeUTF(String s), ...

Control del puntero de acceso:

- seek(long pos) → mueve el puntero a la posición indicada (en bytes).
- getFilePointer() → devuelve la posición actual.
- length() → devuelve el tamaño del fichero en bytes.

Ejemplo básico: escribir y leer con acceso aleatorio

```
import java.io.RandomAccessFile;
import java.io.IOException;

public class EjemploRandomAccess {
    public static void main(String[] args) {
        try (RandomAccessFile raf = new RandomAccessFile("datos.dat", "rw"))
{
            // Escritura
            raf.writeInt(101);          // ocupa 4 bytes
            raf.writeUTF("Pedro");      // escribe cadena en formato UTF
            raf.writeDouble(8.5);       // ocupa 8 bytes

            // Volver al inicio del fichero
            raf.seek(0);

            // Lectura
            int id = raf.readInt();
            String nombre = raf.readUTF();
            double nota = raf.readDouble();

            System.out.println("ID: " + id + ", Nombre: " + nombre + ", Nota:
" + nota);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Observa que aquí no recorremos todo el fichero, sino que controlamos **dónde leer y dónde escribir** gracias a seek().

1.3 Flujos de bytes y flujos de caracteres

En el apartado anterior vimos que existen **formas de acceso** a un fichero:

- **Secuencial** → leer o escribir de principio a fin.
- **Aleatorio** → acceder a posiciones concretas con RandomAccessFile.

Ahora veremos con más detalle las **clases que proporciona Java para implementar los flujos de entrada y salida de datos**, diferenciando entre **texto (caracteres)** y **binario (bytes)**.

1. **Flujos de bytes** → trabajan con datos binarios.
2. **Flujos de caracteres** → trabajan con texto (internamente convierten bytes en caracteres Unicode).

Flujo de bytes

Estos flujos leen y escriben los **bytes en crudo**, sin interpretarlos como texto. Se usan para **archivos binarios**: imágenes, audio, vídeo, PDF, ejecutables.

Por qué no usar Reader/Writer aquí: los *readers/writers* convierten bytes ↔ caracteres (Unicode). Esa conversión **rompería** un PNG o un PDF. Con *byte streams* no hay conversiones: lo que hay en disco es exactamente lo que lees/escribes.

Clases principales

- **FileInputStream/ FileOutputStream** → lectura/escritura de bytes desde un fichero. Casos de uso: copiar imágenes, abrir PDF, mover ficheros binarios.
- **BufferedInputStream/ BufferedOutputStream** → lectura/escritura eficiente de bytes (con buffer). Casos de uso: copiar imágenes, abrir PDF, mover ficheros binarios.
- **DataInputStream/ DataOutputStream** → lectura/escritura de tipos primitivos (int, double, UTF ...).Casos de uso: guardar registros binarios de datos.
- **ByteArrayInputStream** / **ByteArrayOutputStream** → leer/escribir bytes en arrays de memoria, como si fueran un fichero.

Métodos comunes

- read() → lee un byte (-1 si fin de fichero).
- read(byte[] buffer) → lee un bloque de bytes en un array.
- write(int b) → escribe un byte.
- write(byte[] buffer) → escribe un bloque de bytes.
- close() → cierra el flujo.

Clases FileInputStream / FileOutputStream

Qué son: flujos de **bytes** conectados a un **fichero**.

Cuándo: leer/escribir binarios directamente desde/hacia disco.

Constructores útiles:

- new FileInputStream(File|String)
- new FileOutputStream(File|String) y new FileOutputStream(..., /*append*/ true)

Métodos clave:

- int read() (1 byte; -1 si fin)
- int read(byte[] buf) (lee bloque)
- void write(int b), void write(byte[] buf, int off, int len)
- void close()

Ejemplo de escritura con append:

```
try (OutputStream out = new FileOutputStream("log.bin", true)) { // append
    out.write(new byte[]{1,2,3,4});
}
```

Clases BufferedInputStream / BufferedOutputStream

Qué son: *decoradores* que envuelven a un Input/OutputStream y añaden **buffer** en memoria → **menos accesos a disco** → **más rápido**.

Cuándo: casi siempre que leas/escribas ficheros medianos/grandes.

Constructores:

- new BufferedInputStream(InputStream in, int tamBufferOpcional)
- new BufferedOutputStream(OutputStream out, int tamBufferOpcional)

Métodos clave adicionales:

- mark(int readLimit), reset() (si markSupported() es true)
- (flush() en BufferedOutputStream para vaciar el buffer)

Ejemplo lectura con buffer:

```
try (InputStream in = new BufferedInputStream(new
FileInputStream("video.mp4"))) {
    byte[] buf = new byte[16_384]; // 16 KiB
    while (in.read(buf) != -1) { /* procesar */ }
}
```

Clases DataInputStream / DataOutputStream

Qué son: flujos de bytes con métodos para **tipos primitivos** y cadenas en formato **binario portable** (no texto).

Cuándo: cuando quieres guardar/cargar **int/double/boolean/UTF** sin diseñar tu propio formato.

Constructores:

- new DataInputStream(InputStream in)
- new DataOutputStream(OutputStream out)

Métodos clave:

- Lectura: readInt(), readDouble(), readBoolean(), readUTF(), ...
- Escritura: writeInt(int), writeDouble(double), writeBoolean(boolean), writeUTF(String), ...

Ejemplo guardar/leer un registro binario:

```
// Escribir
try (DataOutputStream dos = new DataOutputStream(new
FileOutputStream("registro.bin"))) {
    dos.writeInt(42);
    dos.writeUTF("Ana");
    dos.writeDouble(9.1);
}

// Leer
try (DataInputStream dis = new DataInputStream(new
FileInputStream("registro.bin"))) {
    int id = dis.readInt();
    String nombre = dis.readUTF();
    double nota = dis.readDouble();
}
```

Clases ByteArrayInputStream / ByteArrayOutputStream

Qué son: flujos de bytes **en memoria** (no tocan disco). Útiles para **transformaciones** antes de escribir o para **tests**.

Cuándo: cuando necesitas un buffer dinámico en memoria o simular E/S.

Métodos clave (ByteArrayOutputStream):

- write(...) normal
- byte[] toByteArray() → te devuelve todo lo escrito en un array

Ejemplo componer bytes en memoria y luego guardarlos:

```
ByteArrayOutputStream baos = new ByteArrayOutputStream();
baos.write(new byte[]{10,20,30});
byte[] datos = baos.toByteArray();

try (OutputStream out = new FileOutputStream("salida.bin")) {
    out.write(datos);
}
```

Clases ObjectInputStream / ObjectOutputStream

Qué son: flujos binarios **orientados a objetos** (serialización Java).

Cuándo: cuando guardas **objetos**.

- **Serializar**: convertir un objeto en una secuencia de bytes para poder guardarlo en un fichero o transmitirlo por red.
- **Deserializar**: reconstruir el objeto original a partir de esos bytes.

Es útil para **persistir información** de forma sencilla sin diseñar un formato propio.

Ejemplo

```
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectOutputStream;

public class SerializarEjemplo {
    public static void main(String[] args) {
        Alumno alumno = new Alumno("Pedro", 22);

        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("alumno.obj"))) {
            oos.writeObject(alumno); // serializamos el objeto
            System.out.println("Objeto Alumno guardado en alumno.obj");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

En resumen:

Clase	Uso principal	Métodos comunes	Ventajas / Cuándo usar
FileInputStream	Leer datos binarios desde un fichero	read(), read(byte[]), close()	Útil para abrir y leer ficheros binarios (imágenes, audio, PDF, ejecutables, etc.).
FileOutputStream	Escribir datos binarios en un fichero	write(int), write(byte[]), close()	Ideal para guardar/copiar ficheros binarios. Se puede usar en modo *append*.
BufferedInputStream	Lectura eficiente de bytes con buffer	read(), read(byte[]), mark(), reset(), close()	Envuelve a un InputStream para mejorar el rendimiento (menos accesos a disco).
BufferedOutputStream	Escritura eficiente de bytes con buffer	write(int), write(byte[]), flush(), close()	Envuelve a un OutputStream, acumula en memoria y escribe en bloque.
DataInputStream	Leer datos primitivos (int, double, boolean…) en binario	readInt(), readDouble(), readUTF(), close()	Permite leer datos en tipos nativos sin transformarlos manualmente.
DataOutputStream	Escribir datos primitivos en binario	writeInt(), writeDouble(), writeUTF(), close()	Facilita guardar datos complejos en formato binario de forma portable.
ByteArrayInputStream	Leer datos binarios desde un array en memoria	read(), read(byte[]), available(), close()	Útil para pruebas o cuando los datos ya están en memoria (no en disco).
ByteArrayOutputStream	Escribir datos binarios en memoria (array dinámico)	write(int), write(byte[]), toByteArray(), close()	Permite construir un array de bytes en memoria antes de guardarlo o enviarlo por red.
ObjectInputStream	Leer (deserializar) objetos desde un fichero o flujo	readObject(), close()	Reconstruye objetos previamente guardados con ObjectOutputStream.
ObjectOutputStream	Escribir (serializar) objetos en un fichero o flujo	writeObject(Object o), close()	Guarda objetos completos, no solo datos sueltos, en formato binario.

Flujo de carácteres

Son flujos especializados en trabajar con **texto**. Estos flujos convierten automáticamente los **bytes almacenados en disco** a **caracteres Unicode**, lo que los hace ideales para trabajar con **archivos de texto** (.txt, .csv, .xml, .json).

Diferencia clave con flujos de bytes:

- Los flujos de bytes tratan la información en crudo (binaria).
- Los flujos de caracteres interpretan esa información como **texto**

Las **clases principales de flujo de caracteres** son:

- **FileReader** → lectura carácter a carácter.
- **FileWriter** → escritura carácter a carácter.
- **BufferedReader** → lectura más eficiente y cómoda (línea a línea con readLine()).
- **PrintWriter** → escritura más cómoda (print, println, printf).

Métodos comunes

- read() → lee un carácter (devuelve -1 al llegar al final).
- read(char[] buffer) → lee varios caracteres de golpe.
- write(int c) → escribe un carácter.
- write(String s) → escribe una cadena.
- println(String s) (en PrintWriter) → escribe texto con salto de línea.
- close() → cierra el flujo.

Clase FileReader

Se usa para leer texto plano (.txt, .csv, .xml, .json ...). Lectura carácter a carácter

Convierte automáticamente bytes a caracteres según la codificación del sistema (por defecto UTF-8 o la del SO).

Métodos principales:

- int read() → lee un carácter (devuelve -1 si fin de fichero).
- int read(char[] buffer) → lee varios caracteres y los guarda en un array.
- close() → cierra el flujo.

```
import java.io.FileReader;
import java.io.IOException;

public class LeerTexto {
    public static void main(String[] args) {
        try (FileReader fr = new FileReader("texto.txt")) {
            int c;
            while ((c = fr.read()) != -1) {
                System.out.print((char) c); // convertimos int a char
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Clase FileWriter

Es el complemento de FileReader, para escribir texto carácter a carácter

Permite sobrescribir o añadir (modo append).

Constructores:

- new FileWriter("archivo.txt") → sobrescribe.
- new FileWriter("archivo.txt", true) → modo append (añadir al final).

Métodos principales:

- write(int c) → escribe un carácter.
- write(String s) → escribe una cadena.
- write(char[] buffer) → escribe un bloque de caracteres.
- close() → cierra el flujo.

```
import java.io.FileWriter;
import java.io.IOException;

public class EscribirTexto {
    public static void main(String[] args) {
```

```
        try (FileWriter fw = new FileWriter("salida.txt")) {
            fw.write("Primera línea\n");
            fw.write("Segunda línea\n");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Clase BufferedReader

Es un "decorador" que envuelve un Reader para hacerlo más eficiente. Lectura eficiente línea a línea

Almacena los datos en memoria en un **buffer**, reduciendo accesos a disco.

Su método estrella es **readLine()**.

Métodos principales:

- String readLine() → devuelve una línea completa de texto (sin salto de línea).
- close() → cierra el flujo.

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class LeerLineas {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(new
FileReader("texto.txt"))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                System.out.println(linea);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Clase PrintWriter

Es un "decorador" que envuelve un Writer para ofrecer métodos sencillos como print() y println(). Escritura cómoda

Muy usado en generación de informes o logs en texto.

Métodos principales:

- print(String s) → escribe texto sin salto de línea.

- println(String s) → escribe texto con salto de línea.
- printf(String formato, Object... args) → escribe texto con formato (similar a C).
- close() → cierra el flujo.

```
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;

public class EscribirLineas {
    public static void main(String[] args) {
        try (PrintWriter pw = new PrintWriter(new FileWriter("salida.txt")))
{
            pw.println("Primera línea");
            pw.println("Segunda línea");
            pw.printf("La nota media es %.2f%n", 8.75);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

En resumen:

Clase	Uso principal	Métodos comunes	Ventajas / Cuándo usar
FileReader	Leer texto carácter a carácter desde un fichero	read(), read(char[]), close()	Sencillo para ficheros pequeños de texto. Convierte automáticamente bytes → caracteres.
FileWriter	Escribir texto carácter a carácter en un fichero	write(int), write(String), write(char[]), close()	Útil para generar ficheros de texto. Permite sobrescribir o añadir (*append*).
BufferedReader	Lectura eficiente de texto con buffer	readLine(), read(), ready(), close()	Ideal para leer texto línea a línea. Muy usado en .txt, .csv.
BufferedWriter *(opcional, si lo quieres añadir)*	Escritura eficiente de texto con buffer	write(String), newLine(), flush(), close()	Alternativa a FileWriter cuando necesitas eficiencia y control de líneas.
PrintWriter	Escritura cómoda de texto	print(), println(), printf(), close()	Muy usado para informes, logs o salidas con formato.

ANEXO: Flujos de bytes vs Flujos de caracteres en Java

Característica	Flujos de bytes	Flujos de caracteres
Clase base	InputStream / OutputStream	Reader / Writer
Unidad de trabajo	1 byte (8 bits)	1 carácter (16 bits, Unicode)
Uso principal	Ficheros binarios (imágenes, audio, vídeo, PDF, ejecutables)	Ficheros de texto (.txt, .csv, .xml, .json)
Clases básicas	FileInputStream, FileOutputStream	FileReader, FileWriter
Clases con buffer	BufferedInputStream, BufferedOutputStream	BufferedReader, BufferedWriter
Clases avanzadas	DataInputStream, DataOutputStream (tipos primitivos) ByteArrayInputStream, ByteArrayOutputStream (memoria) ObjectInputStream, ObjectOutputStream (objetos)	PrintWriter (escritura cómoda) CharArrayReader, CharArrayWriter (memoria) StringReader, StringWriter (cadenas en memoria)
Ejemplo típico	Copiar una imagen (FileInputStream → FileOutputStream)	Leer un .csv con BufferedReader línea a línea
Conversión automática	❌ No convierte nada, trabaja con bytes “en crudo”	✅ Convierte automáticamente bytes ↔ caracteres Unicode
Cuándo usar	Siempre que se trate de datos binarios o no interpretables como texto	Siempre que el contenido sea texto legible por humanos

1.4 Introducción a NIO.2 (New Input/Output 2)

Hasta ahora hemos trabajado con la API **clásica de E/S** de Java (java.io), utilizando clases como File, FileReader, FileWriter o BufferedReader.
Aunque siguen siendo muy utilizadas, desde **Java 7** se introdujo una API más moderna y potente llamada **NIO.2** (*New Input/Output 2*), dentro del paquete java.nio.file.

Antes de Java 7, la API principal de E/S era **java.io**, con clases como:

- File, FileReader, FileWriter, BufferedReader...

Java 7 introdujo **NIO.2** (New Input/Output 2) en el paquete **java.nio.file**, más moderna, flexible y potente.

No sustituye totalmente a la API clásica, pero ofrece:

- Mayor rendimiento en algunas operaciones.
- Mejor manejo de rutas y sistemas de ficheros.
- Operaciones atómicas y seguras.
- Soporte multiplataforma más uniforme.

Principales clases y conceptos

1. Path

- Representa una ruta (absoluta o relativa).
- Sustituye a File en la mayoría de usos.
- Se obtiene con **Paths.get(...)**.

```
Path ruta = Paths.get("docs/ejemplo.txt");
System.out.println("Nombre: " + ruta.getFileName());
System.out.println("Directorio padre: " + ruta.getParent());
```

2. Files

- Clase de utilidades estáticas para operar con Path.
- Métodos más comunes:
    - **Crear y borrar**: createFile(), createDirectory(), delete().
    - **Copiar y mover**: copy(), move().
    - **Comprobar**: exists(), isDirectory(), isRegularFile().
    - **Leer y escribir**: readAllLines(), write(), lines() (stream).

```
Path f = Paths.get("ejemplo.txt");
if (Files.notExists(f)) {
    Files.createFile(f);
}
Files.write(f, List.of("Hola", "Mundo"), StandardCharsets.UTF_8);
List<String> lineas = Files.readAllLines(f, StandardCharsets.UTF_8);
```

3. DirectoryStream

- Permite iterar sobre el contenido de un directorio (similar a listFiles()).
- Ventaja: no carga todos los elementos en memoria.

```
Path dir = Paths.get("docs");
try (DirectoryStream<Path> stream = Files.newDirectoryStream(dir)) {
    for (Path p : stream) {
        System.out.println(p.getFileName());
    }
}
```

4. FileVisitor y Files.walkFileTree()

- Para recorrer árboles de directorios de forma recursiva.
- Se implementa una clase que extienda SimpleFileVisitor<Path>.

```
Files.walkFileTree(Paths.get("docs"), new SimpleFileVisitor<>() {
    @Override
    public FileVisitResult visitFile(Path file, BasicFileAttributes attrs) {
        System.out.println("Fichero: " + file);
        return FileVisitResult.CONTINUE;
    }
});
```

Ejemplos comparativos

Con java.io

```
File f = new File("ejemplo.txt");
if (!f.exists()) {
    f.createNewFile();
}
```

Con NIO2

```
Path f = Paths.get("ejemplo.txt");
if (Files.notExists(f)) {
    Files.createFile(f);
}
```

1.5 Manejo de excepciones

¿Por qué son importantes?

- En las operaciones con ficheros **pueden ocurrir muchos errores**: fichero inexistente, permisos insuficientes, disco lleno, rutas inválidas, etc.
- En Java se gestionan con **excepciones comprobadas** (checked exceptions). La más habitual es IOException

Jerarquía básica

- IOException → excepción general de entrada/salida.
    - FileNotFoundException → fichero no encontrado.
    - EOFException → fin de fichero inesperado.
    - UnsupportedEncodingException → codificación de caracteres no soportada.

Formas de gestionar excepciones

1. **try–catch "clásico"**
   Captura la excepción y permite tratarla sin interrumpir el programa.

   Sintaxis:

```
try {
    // Bloque de código que puede lanzar una excepción
} catch (TipoDeExcepcion e) {
    // Código para manejar la excepción
} finally {
    // (Opcional) Bloque que siempre se ejecuta
    // Ideal para liberar recursos (cerrar ficheros, conexiones, etc.)
}
```

   Notas:

   - El bloque catch puede repetirse varias veces para distintos tipos de excepción.
   - El bloque finally es opcional; se ejecuta siempre, ocurra o no una excepción.
   - Si no necesitas liberar recursos, puedes usar solo try { … } catch { … }.

Ejemplo:

```
import java.io.FileReader;
import java.io.IOException;

public class EjemploTryCatchClasico {
    public static void main(String[] args) {
        FileReader fr = null;
        try {
            // Intentamos abrir un fichero de texto
            fr = new FileReader("ejemplo.txt");

            int caracter;
            while ((caracter = fr.read()) != -1) {
                System.out.print((char) caracter);
            }

        } catch (IOException e) {
            // Manejo de errores de entrada/salida (fichero no
encontrado, problemas de lectura, etc.)
            System.err.println("Error de E/S: " + e.getMessage());

        } finally {
            // Cierre manual del recurso para evitar fugas
            if (fr != null) {
                try {
                    fr.close();
                    System.out.println("\nFichero cerrado
correctamente.");
                } catch (IOException e) {
                    System.err.println("Error al cerrar el fichero: "
+ e.getMessage());
                }
            }
        }
    }
}
```

2. **try-with-resources (recomendado desde Java 7)**
   Garantiza que el recurso (fichero, stream) se cierre automáticamente.

   Sintaxis:

```
try (RecursoTipo nombre = new RecursoTipo()) {
    // Bloque de código que usa el recurso
} catch (TipoDeExcepcion e) {
    // Código para manejar la excepción
} finally {
    // (Opcional) Bloque que siempre se ejecuta después
}
```

Notas:

- Todo recurso declarado entre paréntesis debe implementar AutoCloseable (como FileReader, FileWriter, BufferedReader, Scanner, Connection, etc.).
- Al salir del bloque try, los recursos se cierran automáticamente (aunque haya excepción).
- Puedes declarar varios recursos, separados por ;

Ejemplo

```
import java.io.FileReader;
import java.io.IOException;

public class EjemploTryWithResources {
    public static void main(String[] args) {
        // Con try-with-resources no hace falta cerrar el FileReader
manualmente
        try (FileReader fr = new FileReader("ejemplo.txt")) {

            int caracter;
            while ((caracter = fr.read()) != -1) {
                System.out.print((char) caracter);
            }

            System.out.println("\nLectura finalizada.");

        } catch (IOException e) {
            // Manejo de errores de entrada/salida
            System.err.println("Error de E/S: " + e.getMessage());
        }
    }
}
```

Diferencias con el clásico:

- No declaramos FileReader fr = null; fuera del try, lo declaramos directamente en la cabecera del try.
- No hay finally para cerrar el fichero: se cierra solo al terminar el
- Código más corto y menos propenso a errores (no hay que preocuparse de null ni de close()).

3. Propagar la excepción

Cuando no queremos gestionarla en ese método, podemos declararla con throws:

```
public static void leerFichero(String nombre) throws IOException {
    try (FileReader fr = new FileReader(nombre)) {
        // ...
    }
}
```

Buenas prácticas

- Siempre usar **try-with-resources** para evitar fugas (no olvidar cerrar el fichero).
- Dar mensajes de error claros al usuario (ej. *"No se encuentra el fichero alumnos.txt"*).
- Evitar capturar Exception de forma genérica salvo que sea estrictamente necesario.
- Registrar (log) los errores en aplicaciones grande